

Telecom Churn Project Details

1. What dataset is used and where did you get that?

The project uses a synthetically generated Telecom Churn dataset. Instead of relying on a static external dataset (such as the standard IBM Telco dataset), the data is dynamically generated using the ``data_generator.py`` script provided in the project.

This script utilizes Python's Numpy and Pandas libraries to generate realistic customer features (such as tenure, monthly charges, and contract type). It applies realistic business logic to simulate churn probabilities based on factors like sudden price shocks, usage spikes, and frequent tech support calls.

2. What ML algorithms are used?

The core machine learning algorithm used is XGBoost (specifically ``XGBClassifier``), which is highly effective for tabular data and churn prediction.

Additionally, the project integrates SHAP (SHapley Additive exPlanations) via the ``TreeExplainer`` for Explainable AI (XAI). SHAP is used to break down each prediction and identify the specific features contributing to a customer's churn risk, providing transparent reasoning for the model's outputs.

3. What evaluation metrics are used?

The model is evaluated using a comprehensive suite of classification metrics:

- ROC AUC Score (Area Under the Receiver Operating Characteristic Curve) to measure the overall ability to distinguish between churners and non-churners.
- Logloss (Logarithmic Loss), which is used as the primary evaluation metric during the XGBoost training phase.
- Classification Report metrics, including Precision, Recall, and F1-Score, to evaluate the balance of the predictions.
- Confusion Matrix, to visualize True/False Positives and Negatives.

4. Where did you deploy this?

The application is natively containerized using Docker, with both a ``Dockerfile`` and a ``docker-compose.yml`` file included in the repository. This makes it cloud-ready for deployment on any platform that supports Docker containers, such as Render, Heroku, or Hugging Face Spaces.

For local environments, the project is deployed using a Flask development server (or Gunicorn) accessible at ``http://localhost:5000``.

Telecom Churn Project Details

5. How did you make this website?

The website is built as a responsive web dashboard using a lightweight stack:

- Backend/API: Built with Python and the Flask framework to handle routing and serve machine learning predictions.
- Database: TinyDB (a lightweight, document-oriented database) is used to persist high-risk customer data, prediction history, and model drift metrics in JSON format (`at\_risk\_customers.json`).
- Frontend: The user interface is built using HTML, CSS, and JavaScript, served through Flask's Jinja templates from the `templates` directory. This creates an interactive dashboard for monitoring churn metrics and SHAP explanations in real time.